UT2. MANEJO DE CONECTORES

ACTIVIDAD 5

Sentencia DML de modificación de datos sobre una base de datos

Título: Modificación de datos en una Base de Datos MySQL desde Java.

Descripción:

En esta actividad, implementarás una clase **JDBCInsert** en Java que se conectará a tu base de datos MySQL y ejecutará una sentencia **INSERT** para agregar múltiples filas en la tabla **clientes**. La aplicación deberá gestionar posibles errores de conexión y de SQL, y mostrar en la consola el número de filas insertadas exitosamente.

Objetivos:

Desarrollar una aplicación en Java que se conecte a una base de datos MySQL y ejecute sentencias INSERT para agregar múltiples registros en una tabla. Esta actividad te permitirá practicar el uso de sentencias de modificación de datos (DML) en JDBC (Java Database Connectivity), gestionando correctamente las excepciones SQL y verificando el resultado de la operación.

Instrucciones:

1. **Configuración de la Base de Datos:**
    - Utiliza la tabla **clientes** que creaste en la actividad 2, en la base de datos **ut2**. Recuerda que tiene las siguientes columnas:
        - DNI: Identificador único de tipo CHAR(9), no nulo y clave primaria.
        - APELLIDOS: Apellidos del cliente de tipo VARCHAR(32), no nulo.
        - CP: Código postal de tipo CHAR(5), permite valores nulos.
2. **Implementa la Clase JDBCInsert:**
    - En tu proyecto Java, crea una clase llamada JDBCInsert.
    - Define las constantes necesarias (URL, USER, PASSWORD) para gestionar la conexión a la base de datos **ut2**.
3. **Conectar a la base de datos y crear Statement:**
    - Utiliza un bloque try-with-resources para abrir una conexión con la base de datos y crear un objeto Statement. Esta estructura asegura que la conexión y el Statement se cierren automáticamente después de su uso.
4. **Ejecuta una sentencia INSERT para múltiples filas:**

- Utiliza un único INSERT que añada 3 clientes en una sola sentencia: INSERT INTO clientes VALUES (...), (...), (...)
- Al menos uno de los clientes debe tener el CP a NULL.
- La sentencia debe utilizar el método executeUpdate() y almacenar el número de filas afectadas en una variable nFil.
- Muestra en la consola el número de filas insertadas con el mensaje: “Filas insertadas: [nFil]”.

5. **Salida gráfica (opcional):**

- Este apartado no es obligatorio. Si lo realizas, se valorará como funcionalidad extra (nivel 10 de la rúbrica).
- Además del mensaje en la consola, muestra el **número de filas insertadas** en un **cuadro de diálogo usando Java Swing**.
- Para ello, utiliza el método **JOptionPane.showMessageDialog(null, mensaje)**, que abre una ventana emergente con el texto indicado. El primer parámetro (null) indica que el cuadro de diálogo no depende de ninguna otra ventana.
- Los mensajes de error se siguen mostrando siempre en la consola.
- Si realizas este apartado, incluye también una captura del cuadro de diálogo.
- Ten en cuenta que el programa no termina hasta que cierres el cuadro de diálogo. Si no lo ves, puede haber quedado detrás de la ventana de IntelliJ.

6. **Gestión de Excepciones:**

- Implementa un método muestraErrorSQL para mostrar detalles del error en caso de que ocurra una SQLException. Este método debe mostrar:
  - El mensaje de error (getMessage()).
  - El estado SQL (getSQLState()).
  - El código de error del proveedor (getErrorCode()).
- Captura cualquier otra excepción y muestra el mensaje correspondiente en la consola.

7. **Prueba del programa:**
   - Ejecuta el programa y comprueba que en la consola se muestra “Filas insertadas: 3” (y también en el cuadro de diálogo, si has realizado el apartado 5).
   - Ejecútalo una segunda vez y comprueba que, al repetirse los DNI, muestraErrorSQL muestra el mensaje, el estado SQL y el código de error.
   - Comprueba en phpMyAdmin que en la tabla sigue habiendo solo 3 clientes.
   - Incluye una captura de cada ejecución.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Presenta la tarea, el código es correcto, pero no ha demostrado su funcionamiento mediante las capturas de pantalla
7	Presenta la tarea, el código es correcto, demuestra su funcionamiento

	mediante las capturas de pantalla, pero no describe los principales métodos incluidos en el código.
9	Se completa la tarea correctamente en todos sus apartados
10	Se implementa una funcionalidad extra o un detalle innovador que mejora el programa.

Entrega:

1. **Incluye aquí el código fuente**:

```
package org.example;

import javax.swing.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

/**
 * @author Juan Gabriel Galarza Claros
 */
public class JDBCInsert {

// Constantes de conexión a la base de datos
private static final String URL = "jdbc:mysql://localhost:3306/ut2";
private static final String USER = "dam";
private static final String PASSWORD = "123456";

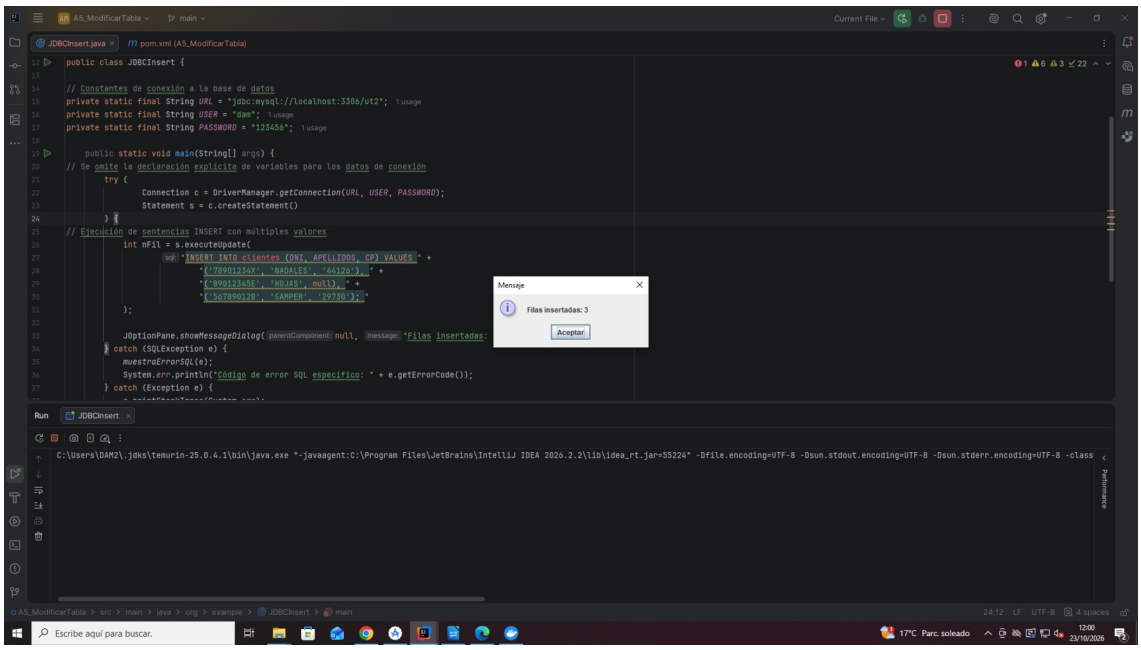
    public static void main(String[] args) {
// Se omite la declaración explícita de variables para los datos de conexión
        try (
                Connection c = DriverManager.getConnection(URL, USER, PASSWORD);
                Statement s = c.createStatement()
        ) {
// Ejecución de sentencias INSERT con múltiples valores
            int nFil = s.executeUpdate(
                    "INSERT INTO clientes (DNI, APELLIDOS, CP) VALUES " +
                            "('78901234X', 'NADALES', '44126'), " +
                            "('89012345E', 'HOJAS', null), " +
                            "('56789012B', 'SAMPER', '29730'); "
            );

            JOptionPane.showMessageDialog(null, "Filas insertadas: " + nFil);
        } catch (SQLException e) {
            muestraErrorSQL(e);
            System.err.println("Código de error SQL específico: " + e.getErrorCode());
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }

public static void muestraErrorSQL(SQLException e) {
```

```
    System.err.println("--- Se ha producido un error SQL ---");
    System.err.println("Mensaje: " + e.getMessage());
    System.err.println("Estado SQL: " + e.getSQLState());
    System.err.println("Código de error del proveedor: " + e.getErrorCode());
    System.err.println("-----------------------------------");
}
}
```

2. **Incluye aquí las capturas de pantalla que demuestren el correcto funcionamiento del programa desarrollado**:



3. **Describe aquí los métodos y variables principales utilizadas en el programa**:

Clases

- **JDBCInsert (Clase principal del programa):** Es la clase contenedora principal donde se escribe todo el código ejecutable y los métodos auxiliares.
- **Clases de librerías externas utilizadas:**
  - **Connection:** Clase para gestionar la conexión con la base de datos.
  - **Statement:** Clase para ejecutar las instrucciones SQL.
  - **SQLException:** Clase para capturar errores específicos de SQL.

- **JOptionPane:** Clase gráfica de Java para mostrar ventanas de aviso emergentes.

Métodos

- **main(String[] args):** El **método principal** y punto de partida del programa. Su función es conectarse a la base de datos, ejecutar el comando INSERT para añadir los datos, mostrar el resultado en una ventanita y capturar posibles errores.

- **muestraErrorSQL(SQLException e):** Un método auxiliar diseñado específicamente para imprimir por la consola los detalles detallados si ocurre un fallo con la base de datos.

Variables

- **URL**: Dirección y puerto de la base de datos.

- **USER**: Nombre de usuario para entrar a la base de datos.

- **PASSWORD** : Contraseña del usuario.

- **c** *(Variable/Objeto Connection)*: Guarda la **conexión activa** abierta con la base de datos.

- **s** *(Variable/Objeto Statement)*: Sirve para **enviar la orden SQL** a la base de datos.

- **nFil** *(Variable int)*: Almacena el **número de filas** que se acaban de insertar correctamente.

- **e** *(Variable Exception/SQLException)*: Captura y guarda el **error** si algo sale mal durante la ejecución.